SLIDRONIX

Phase H → Frontend, Data Flow, Backend/Database Status & Deployment Review

Final Presentation Companion • 21 October 2026

A source-grounded review of the implemented Phase H risk-prediction and dashboard pipeline, with a clear separation between confirmed implementation and items that need repository verification.

Important: The supplied Phase H documentation confirms Python inference/output scripts and a React + Leaflet dashboard consuming prepared prediction data. It does not establish that a separate API backend, persistent database, or public deployment exists. Verify the repository and actual demo before claiming these components are implemented.

1. Executive Summary

Slidronix is an AI-based landslide-risk mapping prototype focused on the Western Ghats study region. It combines historical landslide locations with terrain, satellite-derived vegetation/land-cover, soil, and rainfall features. The model pipeline produces a probability for a landslide/background classification, and Phase H turns those predictions into Low, Moderate, and High categories for GIS-style visualization.

The Phase H reference confirms that a trained multimodal Fusion checkpoint is loaded by a Python script, predictions are exported to CSV, a GIS risk map is saved as PNG, and a React + Leaflet dashboard displays prepared prediction data. The dashboard supports satellite/street basemaps, risk-level filtering, point selection, and selected-location details.

For the presentation, describe this as a research prototype for spatial risk visualization—not as a fully operational real-time warning service. The documented rainfall sequence is fixed to 1–30 January 2022; live rainfall APIs and real-time alerts are future enhancements.

2. End-to-End System Architecture

Layer / stage	Confirmed role	Implementation evidence
Data preparation	Build aligned environmental inputs and test rows	Phase E test dataset; 26 tabular features; terrain patches; 30-day rainfall sequence
Model inference	Generate a probability for each aligned sample	Python script: generate_phase_h_risk_predictions.py; trained Fusion checkpoint
Risk categorization	Map probability into Low / Moderate / High	Implemented in the prediction pipeline; thresholds are project-defined
Prediction output	Export generated predictions	outputs/phase_h/slidronix_phase_h_risk_predictions.csv
GIS output	Render spatial points by risk level	outputs/phase_h/slidronix_phase_h_risk_map.png
Frontend dashboard	Explore risk points interactively	React + Leaflet; frontend/src/App.jsx and App.css
API backend	Not established by supplied documentation	Do not claim a REST API unless source code and a running endpoint are verified
Database	Not established by supplied documentation	Documented flow uses CSV → JSON → React; no database engine/schema is named
Deployment	Not established by supplied documentation	Claim deployment only if a live URL or hosting configuration is verified

Simplified architecture diagram

Historical inventory + environmental datasets
↓
Preprocessing and aligned Phase E samples
↓
Trained Fusion model (MLP tabular encoder + CNN terrain encoder + rainfall Transformer)
↓
Probability 0–1 → **Low / Moderate / High**
↓
Prediction CSV → JSON data file → **React + Leaflet dashboard**
■ GIS risk map PNG

3. Phase H: What Was Implemented

3.1 Multimodal Fusion model

The model uses three input branches. The tabular MLP encodes 26 environmental features into a 32-dimensional representation. The CNN encodes a 2 × 32 × 32 elevation/slope patch into 32 dimensions. The rainfall Transformer processes 30 daily rainfall values and produces another 32-dimensional representation. These are concatenated into a 96-dimensional vector and passed through a classification head. A sigmoid converts the output logit into a value between 0 and 1.

3.2 Prediction coverage and output

Item	Documented result
Phase E test rows	2,800
Valid CNN terrain-patch samples	2,761
Aligned Phase H predictions	2,761
Probability range	0.0001–0.9978
Mean probability	0.4974

The model produces predictions for the common aligned subset because terrain patches are not available for every test row. Do not say Phase H covers all 2,800 rows; the documented count is 2,761.

3.3 Risk distribution

Risk class	Count	Share
Low	1,083	39.22%
Moderate	490	17.75%
High	1,188	43.03%

Risk classes are used for display and exploration. The supplied documentation says the thresholds are project-defined and are not validated emergency thresholds.

4. Frontend: React + Leaflet Dashboard

The Phase H documentation states that the dashboard is implemented in React and Leaflet. It uses satellite imagery as its default basemap and offers a street-map alternative.

• **Map visualization:** displays prediction points using latitude and longitude.

• **Risk-level filtering:** lets users show selected risk categories.

• **Point selection:** selecting a point exposes its details.

• **Location details:** displays risk class, probability, coordinates, and selected environmental features.

• **Files:** frontend/src/App.jsx is the main UI component; frontend/src/App.css contains dashboard styling.

> **Presentation phrasing:** “The frontend is a geospatial visualization interface. React manages the interface, while Leaflet renders the interactive map and map layers. The dashboard reads prepared prediction data and lets the user inspect spatial risk patterns.”

Verify before presenting: show the actual data-loading code that loads the JSON file. Do not invent a separate backend API if the frontend reads a static JSON file.

5. Backend and Data-Storage Status

5.1 What the documents establish

The Phase H documentation names generate_phase_h_risk_predictions.py for inference and risk classification, and create_phase_h_risk_map.py for creating the PNG risk map. It also shows a dashboard flow of prediction CSV → risk_predictions.json → React + Leaflet dashboard.

5.2 Is there a backend server?

The supplied PDFs do not identify a Flask, FastAPI, Django, Node/Express, or other web API server, nor do they document API routes or request/response schemas. Python inference scripts are part of the processing pipeline, but that alone does not prove a web backend server exists.

5.3 Is there a database?

The supplied PDFs do not name a relational or NoSQL database, a schema, tables/collections, migrations, or database connection settings. The documented artifacts are CSV/JSON files and a PNG map. Based on these documents alone, describe the current flow as file-based data exchange.

If the repository contains a database or API, update this section only after verifying source code/configuration. Be ready to show the database name, schema, connection method, API endpoints, and stored information.

5.4 Useful terminology

Term	Meaning
Frontend	The user-facing interface; here, the React + Leaflet dashboard.
Inference pipeline	Code that loads a trained model and calculates predictions.
Backend API	A server exposing endpoints for the frontend or other clients; not confirmed by the supplied PDFs.
JSON	A structured text format for exchanging data between software components.
CSV	A tabular text format used for prediction rows and attributes.
Database	A managed data store queried/updated through a database engine; not confirmed by the supplied PDFs.
Deployment	Making the application available in a target environment; public hosting is not confirmed by the supplied PDFs.

6. Data Pipeline and Key Files

File / path	Purpose
src/generate_phase_h_risk_predictions.py	Loads Fusion checkpoint, runs inference, creates probability/risk-class predictions
src/create_phase_h_risk_map.py	Creates a GIS-style PNG map from spatial predictions
outputs/phase_h/slidronix_phase_h_risk_predictions.csv	Prediction rows and spatial/risk values
outputs/phase_h/phase_h_risk_summary.txt	Risk prediction summary
outputs/phase_h/slidronix_phase_h_risk_map.png	Generated map image
outputs/phase_h/phase_h_risk_map_summary.txt	Map generation summary
frontend/src/App.jsx	React dashboard component
frontend/src/App.css	Dashboard styles
risk_predictions.json	JSON data file shown in the documented flow; verify its actual path and generation script in the repository

7. Deployment: What to Say and What to Verify

The PDFs describe implementation and output files but do not specify a public URL, hosting provider, container image, web server, CI/CD pipeline, domain, or production deployment configuration. Do not claim cloud deployment unless you can demonstrate it.

Before the demo

• Run the Python inference/output pipeline where its dependencies and model checkpoint are available.

• Confirm the CSV, JSON, and PNG outputs exist and are up to date.

• Start the React frontend using the repository's actual package scripts; check package.json rather than guessing the command.

• Verify the dashboard loads the intended JSON and map tiles load with an internet connection.

• Test risk filters and select map points to show risk class, probability, coordinates, and environmental attributes.

• Keep a screenshot of the working dashboard and generated risk map as a backup.

If asked how it is deployed

A safe answer based on current documentation is: “We have implemented the prediction and dashboard pipeline and can run the frontend in the project environment. Public/cloud deployment details are not part of the documented implementation yet.” Use this only if it matches the actual demo state.

8. Important Scientific Limitations

• **Not live early warning yet:** rainfall is a fixed 1–30 January 2022 reference period, not a live feed or event-aligned pre-landslide sequence.

• **Sampled background locations:** these are not guaranteed locations with no historical landslide ever.

• **Study boundary:** derived from administrative boundaries and the historical inventory; do not call it an official Western Ghats boundary.

• **Coverage:** Phase H predictions cover 2,761 aligned samples because 39 test rows lack valid CNN patches.

• **Risk thresholds:** Low/Moderate/High cutoffs are project-defined, not validated emergency thresholds.

• **External generalization:** independent geographic and temporal validation is needed before operational use.

A responsible description is: “Slidronix is a prototype for landslide susceptibility/risk estimation and spatial visualization. Real-time operational warning requires live environmental data integration, event-aligned rainfall, validated thresholds, and field/agency validation.”

9. Common Viva Questions and Short Answers

Q. What problem does Slidronix address?

A. It explores how historical landslide records and environmental features can be combined to estimate spatial landslide risk and visualize it for the Western Ghats study region.

Q. Why use multiple data sources?

A. Terrain, vegetation/land cover, soil, and rainfall describe different aspects of the environment.

Q. Why use an MLP?

A. It learns nonlinear relationships from structured tabular environmental features.

Q. Why use a CNN?

A. It learns local patterns from 32 × 32 terrain patches with elevation and slope channels.

Q. Why use a Transformer?

A. Self-attention models relationships among ordered daily rainfall observations.

Q. Why fuse the branches?

A. The branches encode different modalities; their embeddings are combined so the classifier can use complementary information.

Q. What is the Fusion model's ROC-AUC?

A. 0.9122 on its 2,761-sample valid spatial test subset. It is a ranking metric, not 91.22% accuracy.

Q. Why are there only 2,761 Phase H predictions?

A. Only 2,761 of the 2,800 test rows had valid CNN terrain patches for aligned multimodal inference.

Q. What does a probability of 0.8 mean?

A. The model assigns a score of 0.8 to the positive class; it should not automatically be treated as a calibrated real-world 80% event probability.

Q. Is this a real-time early-warning system?

A. Not yet. Rainfall is fixed to January 2022 and live environmental APIs/real-time alerts are future enhancements.

Q. What is the backend?

A. The documents confirm Python inference scripts and a React/Leaflet frontend reading prepared data, but do not establish a web API backend.

Q. Which database did you use?

A. The supplied documentation does not specify a database; it documents CSV/JSON output files.

Q. Where is it deployed?

A. The PDFs do not document public/cloud deployment. State the actual demo environment only after verifying it.

Q. What is the main future work?

A. Integrate current rainfall/environmental APIs, support user-selected coordinates, generate fresh inference, validate thresholds, and build alerting after proper evaluation.

10. A 60–90 Second Presentation Script

"Slidronix is an AI-based landslide-risk mapping prototype for the Western Ghats study region. We extracted and cleaned a historical landslide inventory, then constructed positive and sampled-background points. We enriched these locations with terrain elevation and slope, MODIS vegetation and land-cover information, SoilGrids soil properties, and CHIRPS rainfall features. We trained and compared a tabular MLP, a terrain CNN, a rainfall Transformer, a multimodal Fusion model, and a Random Forest benchmark. The Fusion model combines 32-dimensional representations from the tabular, spatial, and rainfall branches into a 96-dimensional representation and outputs a landslide score. In Phase H, we generated predictions for 2,761 aligned test locations, assigned project-defined Low, Moderate, and High categories, created a GIS risk map, and built a React + Leaflet dashboard for interactive exploration. The current version uses a fixed January 2022 rainfall reference period, so live rainfall-triggered warning is future work rather than a current capability."

11. Last-Minute Checklist

• Know the objective and complete A → H pipeline.

• Memorize Phase H coverage: 2,761 aligned predictions out of 2,800 test rows.

• Know the three model branches and why they are fused.

• Know key results: Fusion ROC-AUC 0.9122; MLP F1 0.8333; CNN recall 0.8831; RF ROC-AUC 0.9093.

• Open the dashboard and click a point; show risk class, probability, coordinates, and feature details.

• Show the generated risk map PNG and prediction CSV/JSON if asked about outputs.

• Do not claim a database, REST API, cloud deployment, live rainfall, or emergency alerts unless you can demonstrate each one.

• Be honest about limitations.

Document basis: the four supplied Slidronix PDFs (Phases A–B, C–E, F–G, and H). This companion summarizes their documented implementation and flags backend/database/deployment details that those PDFs do not establish.